PRODUCT + INTEGRATION BRIEF

Event Cards

Reuse the existing Pixl API. Keep email sign-in. Don’t duplicate the system.

Existing API is source of truth | **Email sign-up / sign-in only** | **No Google sign-in for now**

Decision to carry forward

Build the new experience as a client of the current Pixl Sports API. The API already owns Event Card persistence, owner access, public links, Calendar-link verification, and deletion behavior.

What an Event Card is

An Event Card is a shareable page for **one soccer game or one practice**. It is an event-specific family view that can connect preparation, live stats, and the athlete’s HYPE Card. It is not a general calendar, multi-event schedule, or replacement for the Calendar.

One event, one card

A card contains exactly one `game` or `practice`. Games need a date and opponent; practices need a date. Preserve the event’s local time and IANA time zone.

Two ways to create

Create a card independently, or link it to an event in the owner’s Calendar. The relationship must be verified by the server.

Public, but limited

The public view contains basic event details and only related destinations the owner has deliberately shared.

Terminology boundary

Use **Event Card(s)** in UI copy. Keep established “Game On” route names, storage keys, and API values intact for compatibility.

Implementation principle

Reuse behavior and contracts rather than copying backend logic. Keep the new project’s UI focused on consuming the API safely.

USE THE EXISTING ROUTES

API contract

The route prefix shown below is `/api`. The current OpenAPI specification and generated React client are the canonical contract.

Purpose	Method + path	Access
Create Event Card	**POST** `/api/schedule-shares`	Signed in; send `purpose: "game-on-card"`
Update Event Card	**PATCH** `/api/schedule-shares/:shareToken`	Owner only; keep the card purpose
List owner's cards	**GET** `/api/game-on-cards/owned`	Signed in
Delete owner's card	**DELETE** `/api/game-on-cards/owned/:shareToken`	Owner only; response says if Calendar event was also deleted
Read public card	**GET** `/api/game-on-cards/:shareToken`	Public; no sign-in

Do not use the old public schedule-share route

`/api/schedule-shares/:shareToken` is not the public Event Card endpoint. Public cards must use `/api/game-on-cards/:shareToken`, which returns the filtered card response.

Authentication: email only

- Offer email sign-up/sign-in only for the initial release; do not add Google sign-in yet.
- The owner routes require a Clerk-authenticated session that the existing API accepts.
- Use the same Clerk app/tenant trusted by this API. Owner identity is the verified Clerk user ID—not the email string.
- A separate Clerk tenant can issue a different user ID for the same email, so that account will not automatically own the existing user's cards.
- Confirm email sign-in is enabled for the Clerk instance used by the new app. Keep the public card route public.

Client and environment

Reuse `lib/api-spec/openapi.yaml` and, where the new project can consume it, `lib/api-client-react`. Configure the correct API origin per environment. Don't hardcode a development URL, expose Clerk secrets, or maintain duplicate request/response types.

PRESERVE THESE BOUNDARIES

Calendar linking, privacy, and identity

Linked event edits

When a card is linked to a Calendar event, save the Calendar edit first and then update the card. Keep the card's public token stable. Report partial failure so the owner can retry without creating another share.

Safe deletion

Delete the linked Calendar event only when the API verifies the owner, event ID, Calendar project, and unique relationship. If the match is missing, ambiguous, or unverified, leave Calendar unchanged and report that outcome.

Private preparation

Prep notes and checklist contents stay private unless the owner separately shares Prep. Creating or updating an Event Card must not publish Prep automatically.

Separate HYPE visibility

Showing event listings on the HYPE Card is controlled separately by "Allow events." A public Event Card link can exist without making the owner's full event list visible there.

Only share what the owner chose

- The public response includes one sanitized event and the current states of related shared destinations.
- Stats require a separately shared tracker destination. Do not expose raw tracker data or infer that private stats are public.
- Prep is marked private unless there is a separately shared checklist. HYPE Page visibility is also reported separately.
- Do not include Calendar snacks, drinks, notes, or other private planning fields in the public Event Card payload.

Legacy identity rule — important

Never decide that a share is an Event Card solely because it contains one game or practice. Calendar sharing used the same underlying storage shape. The owner collection uses an explicit internal marker; that marker must never appear in public responses. Preserve older public card links, but don't automatically promote ambiguous old shares into the owner's card collection.

Durable data, not local-only ownership

The API owns the durable card and its owner relationship. Browser storage supports local details and compatibility with older device-saved cards; it is not the cross-device source of truth. Don't use local storage to decide what the server says the user owns.

Locker connection

Successful Event Card creation credits the Locker milestone ledger using the event ID. Retries and edits should not award duplicate progress; deletion does not erase earned activity.

TIME-AWARE BEHAVIOR

Event timing and reminders

Card stage

Show “Starting Soon” during the one-hour pregame window; switch to “Live” at the scheduled start in the event’s saved time zone. Tracker capture may be available before the card becomes Live.

Daylight-saving transitions

For a nonexistent spring-forward local time, shift forward through the gap. For a repeated fall-back time, use the earlier instant. Use this same resolved start for the badge and pregame window.

One-hour reminder

Uses the event’s saved time zone. It is a separate, account-level opt-in.

Day-of reminder

Scheduled for 7:00 a.m. in the account’s saved alert time zone. Missing saved choices default off; app installation does not opt someone in.

Delivery is not automatic just because the UI exists

Scheduled alerts need an active, configured server-side caller. Show deployment readiness honestly. A manual follower announcement is a separate owner-confirmed action; never turn it into an automatic send.

WHERE TO START

Code landmarks

These files show the current UI, API rules, and shared contract. The new project should consume the API rather than copy its backend implementation.

`artifacts/league-matrix-studio/src/features/game-on-card.tsx`

Create/edit form and public card page

`artifacts/league-matrix-studio/src/features/game-on-dashboard-collection.tsx`

Owner card collection and delete feedback

`artifacts/league-matrix-studio/src/features/game-on-card-storage.ts`

Device-local and legacy compatibility

`artifacts/api-server/src/routes/scheduleShares.ts`

Validation, ownership, public read, and safe Calendar deletion

`artifacts/api-server/src/jobs/gameOnReminderQueue.ts`
.
`artifacts/api-server/src/jobs/plannerNotifications.ts`

Reminder scheduling and delivery

`lib/api-spec/openapi.yaml`
.
`lib/api-client-react`

Shared contract and generated client

Acceptance checklist

- [ ] Email-only sign-up/sign-in; no Google option for now.
- [ ] Owner requests work only with the same Clerk identity trusted by the API.
- [ ] Create, update, list, delete, and public read use the existing routes.
- [ ] A card has one game or practice; time zone is preserved and validated.
- [ ] Public links reveal only event details and destinations the owner chose to share.
- [ ] Calendar deletion is verified and unique; ambiguous links leave Calendar intact.
- [ ] One-event legacy shares are not automatically treated as Event Cards.
- [ ] Reminder consent, account zone, event zone, and scheduler readiness are respected.

> **Non-negotiable boundary**
>
> Keep the API authoritative for owner identity and persisted cards. Preserve its privacy filtering, legacy-card behavior, and safe Calendar deletion rather than approximating them in the client.